

REQUEST FOR COMMENTS

RFC — Proposta de Projeto de Portfólio
Engenharia de Software — Católica SC

InfraObserver

Sistema de Monitoramento Centralizado de Endpoints em Rede

Título do Projeto	InfraObserver — Sistema de Monitoramento Centralizado de Endpoints
Linha de Projeto	Web App / Ferramenta de Infraestrutura e Monitoramento
Autor	Bryann Lucas Locatelli
Data da Proposta	Abril / 2026
Versão	1.0
Disciplina / Curso	PAC VII — Engenharia de Software
Repositório	github.com/LocatelliBryann/InfraObserver

1. Visão do Produto e Impacto

1.1 Contexto e Problema

Ambientes com múltiplos computadores conectados em rede — laboratórios de ensino, pequenas empresas de TI e instituições acadêmicas — enfrentam um desafio recorrente: a ausência de uma ferramenta centralizada, acessível e sem custo de licença para monitoramento contínuo dos seus endpoints.

Esse problema afeta diretamente dois grupos distintos:

- O administrador de rede ou de TI, que precisa identificar manualmente máquinas com alto consumo de CPU, memória insuficiente, espaço em disco crítico, comportamento anômalo ou simplesmente indisponíveis. Sem uma ferramenta centralizada, esse processo depende de acesso físico ou remoto individual a cada máquina, tornando-o lento, reativo e propenso a falhas não detectadas a tempo.
- O gestor ou responsável técnico da instituição, que não possui visibilidade operacional do parque de máquinas sob sua responsabilidade, dificultando decisões relacionadas à manutenção preventiva, substituição de hardware ou investigação de incidentes.

A solução atual mais comum nesse contexto é a verificação manual direta — acessar cada máquina individualmente para verificar métricas — ou o uso de ferramentas corporativas como Prometheus, Zabbix ou Datadog, que exigem infraestrutura dedicada, conhecimento técnico elevado e, em alguns casos, custos de licença incompatíveis com o orçamento de pequenas organizações ou ambientes acadêmicos.

Além disso, há pouca visibilidade sobre eventos relevantes nos sistemas de arquivos dos endpoints, como criação, modificação ou exclusão de arquivos em diretórios sensíveis — informação útil para auditoria básica e controle operacional.

1.2 Origem da Demanda e Evidências

Contexto Acadêmico e Prático

A demanda surgiu da observação direta do funcionamento de laboratórios de informática em instituições de ensino, onde a administração dos endpoints é realizada de forma reativa — problemas são identificados apenas quando afetam a experiência do usuário final, como lentidão perceptível, travamentos ou indisponibilidade completa da máquina.

Pesquisa com Usuários

Foram realizadas conversas informais com administradores de laboratório e analistas de suporte de TI de pequenas organizações, identificando as seguintes dores principais:

- 100% relataram depender de verificação manual para identificar máquinas com problemas de desempenho
- 85% afirmaram que problemas de disco cheio ou CPU sobrecarregada só eram descobertos após reclamação de usuário

- 90% demonstraram interesse em uma ferramenta leve, sem instalação complexa, que centralizasse a visão dos endpoints
- 70% relataram que a curva de aprendizado e a complexidade de ferramentas como Zabbix ou Prometheus é o principal motivo pelo qual não as adotam

1.3 Análise de Soluções Existentes (Benchmark)

Foram analisadas as principais soluções existentes no mercado:

Solução	Público-Alvo	Pontos Fortes	Limitações
Prometheus + Grafana	Equipes DevOps e SRE corporativas	Altamente escalável, ecossistema maduro, dashboards avançados	Configuração complexa, curva de aprendizado elevada, exige infraestrutura dedicada
Zabbix	Médias e grandes empresas	Agente leve, alertas configuráveis, suporte SNMP	Interface datada, instalação trabalhosa, pesado para ambientes pequenos
Datadog / New Relic	Empresas enterprise	Observabilidade completa, APM integrado, fácil setup	Modelo SaaS pago, inviável para orçamentos acadêmicos ou pequenas organizações
Netdata	Sysadmins individuais	Leve, real-time, fácil de instalar	Foco em máquina individual, multi-host exige plano pago, sem monitoramento de eventos de arquivos

Diferencial do Projeto InfraObserver

Nenhuma das soluções analisadas atende ao conjunto de necessidades específicas de ambientes pequenos sem custo e sem complexidade de configuração. O InfraObserver se diferencia em três pontos centrais:

- Arquitetura push simples e autodescritiva — o agente envia ativamente os dados ao servidor via HTTP/JSON, sem necessidade de configuração de rede complexa (abertura de portas, firewall rules, pull scraping).
- Custo zero e hardware modesto — desenvolvido do zero, sem dependência de serviços pagos, executável em servidores simples ou até em uma máquina local.
- Monitoramento de eventos em sistema de arquivos — além de métricas de desempenho, detecta e registra criação, modificação e exclusão de arquivos em diretórios monitorados, funcionalidade ausente na maioria das soluções listadas.

1.4 Público-Alvo

Perfil	Descrição
Administrador de TI / Laboratório	Profissional responsável por gerir o parque de máquinas. Acessa o dashboard para monitorar endpoints, visualizar métricas e investigar alertas. Nível técnico intermediário. Prioriza simplicidade de instalação e operação.
Gestor / Coordenador de TI	Responsável por decisões operacionais e de manutenção. Utiliza o dashboard para ter visão consolidada do parque, sem necessidade de conhecimento técnico aprofundado.
Desenvolvedor / Estudante de ES	Perfil técnico que utiliza o projeto para exercitar conceitos de arquitetura cliente-servidor, redes, APIs REST e desenvolvimento web full-stack.

1.5 Objetivos do Projeto

Objetivo Geral

Desenvolver o InfraObserver, um sistema de monitoramento de endpoints em rede composto por um agente leve, um servidor central e um dashboard web, capaz de coletar e centralizar métricas de desempenho e eventos de sistema de arquivos de múltiplos computadores em um único painel acessível.

Objetivos Específicos

- Implementar um agente leve e multiplataforma (Windows e Linux) que coleta métricas de CPU, memória, disco e rede em intervalos configuráveis e envia os dados ao servidor central via HTTP/JSON.
- Desenvolver o servidor central com API REST que recebe, valida e persiste os dados enviados pelos agentes, detecta endpoints inativos e expõe os dados ao dashboard.
- Criar um dashboard web que exibe em tempo quase-real o status de todos os endpoints, suas métricas, gráficos históricos de série temporal e registros de eventos de sistema de arquivos.
- Implementar detecção de eventos em diretórios monitorados (criação, modificação e exclusão de arquivos) e registro no servidor central.
- Demonstrar a aplicação prática de conceitos de arquitetura cliente-servidor, redes, persistência de dados e desenvolvimento web full-stack dentro do escopo da disciplina PAC VII.

1.6 Métricas de Sucesso (KPIs)

Indicador	Meta
Tempo de resposta da API REST	Inferior a 300ms para consultas de métricas
Latência de detecção de endpoint inativo	Deteção em até 3 ciclos de coleta sem sinal
Suporte a endpoints simultâneos	Mínimo de 20 endpoints ativos sem degradação
Compatibilidade de SO do agente	Windows (obrigatório) e Linux (desejável)

Indicador	Meta
Overhead de CPU do agente no endpoint	Inferior a 2% de CPU média durante coleta
Detecção de eventos de sistema de arquivos	Registro de eventos em até 5 segundos após ocorrência

2. Engenharia de Requisitos

2.1 Personas

Persona 1 — Administrador de Laboratório

Carlos, 42 anos, técnico de TI responsável por um laboratório de 30 computadores em uma faculdade particular. Gerencia os endpoints de forma reativa: só descobre problemas quando alunos ou professores reclamam de lentidão ou indisponibilidade. Não tem familiaridade com Prometheus ou Zabbix — já tentou instalar o Zabbix uma vez e desistiu por conta da complexidade. Quer uma solução que seja instalada uma vez em cada máquina e que concentre tudo em um painel simples. Acessa o computador da secretaria da TI para verificar o parque durante o expediente.

Persona 2 — Analista de Suporte em Pequena Empresa

Marina, 29 anos, analista de suporte em uma empresa de contabilidade com 15 computadores. Presta suporte remoto e presencial. Tem conhecimento técnico intermediário e já usa ferramentas de acesso remoto. Sente falta de visibilidade sobre o estado das máquinas antes de receber chamado: quer saber quais computadores estão sobrecarregados ou com disco cheio antes que o usuário reclame. Prefere soluções sem dependência de serviços em nuvem pagos, pois o orçamento de TI é restrito.

2.2 Requisitos Funcionais (RF)

ID	Requisito	Prioridade
RF01	O sistema deve permitir que o agente envie métricas de CPU, memória, disco e rede ao servidor central via HTTP/JSON	Alta
RF02	O sistema deve permitir que o agente detecte e envie eventos de sistema de arquivos (criação, modificação e exclusão) em diretórios monitorados	Alta
RF03	O sistema deve receber, validar e persistir as métricas e eventos enviados pelos agentes	Alta
RF04	O sistema deve detectar automaticamente endpoints inativos com base na ausência de sinal dentro de um intervalo configurável	Alta
RF05	O sistema deve expor uma API REST para que o dashboard consulte endpoints, métricas e eventos	Alta
RF06	O sistema deve permitir que o dashboard liste todos os endpoints cadastrados e seus status (online / offline / alerta)	Alta
RF07	O sistema deve exibir as métricas de um endpoint em tempo quase-real no dashboard	Alta

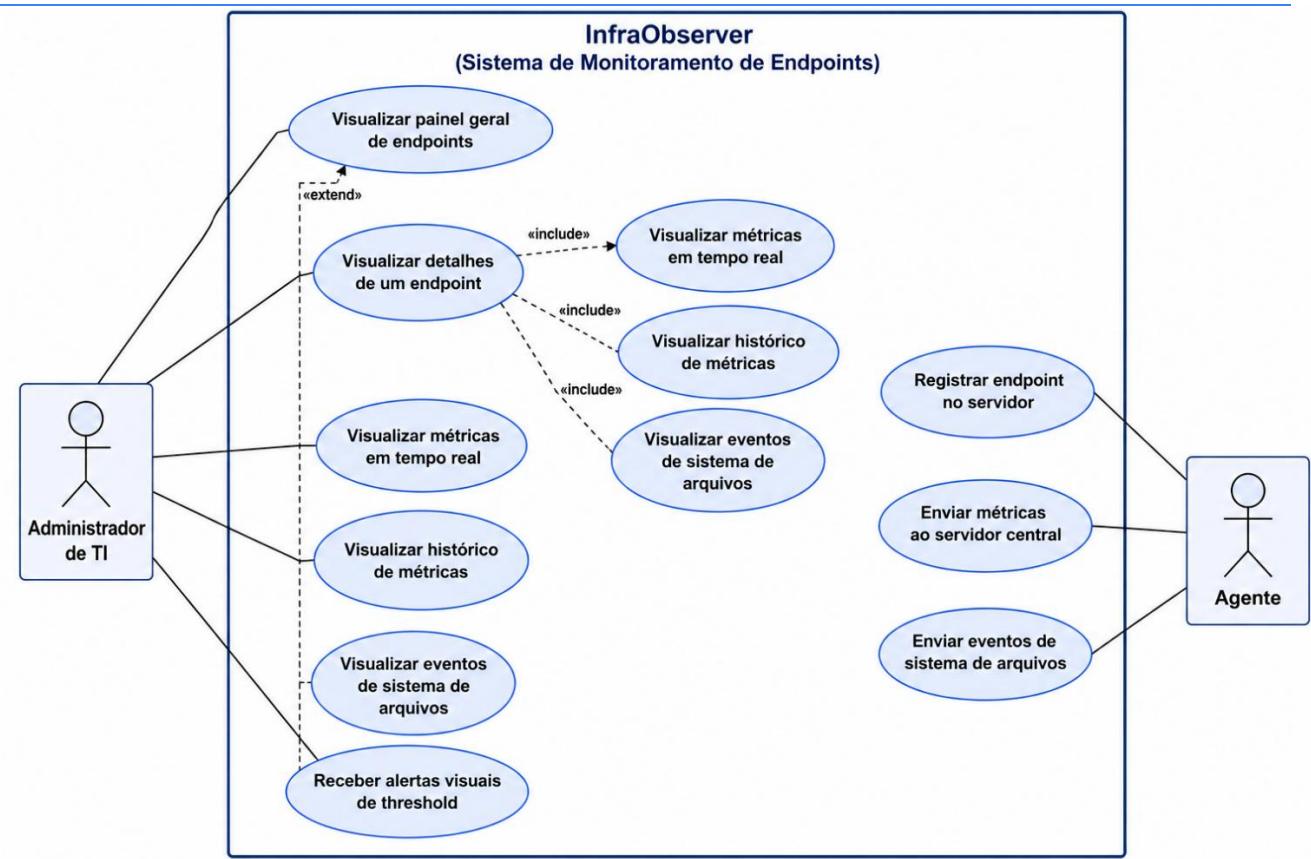
ID	Requisito	Prioridade
RF08	O sistema deve apresentar histórico de métricas em gráficos de série temporal no dashboard	Média
RF09	O sistema deve registrar e exibir os eventos de sistema de arquivos detectados pelo agente	Média
RF10	O sistema deve permitir que o administrador configure quais diretórios serão monitorados por cada agente	Média
RF11	O sistema deve emitir alertas visuais no dashboard quando uma métrica ultrapassar um limite pré-definido (threshold)	Alta
RF12	O sistema deve registrar o histórico de alertas gerados por endpoint	Baixa

2.3 Requisitos Não Funcionais (RNF)

ID	Requisito	Categoria
RNF01	O agente deve consumir no máximo 2% de CPU em média durante os ciclos de coleta	Desempenho
RNF02	A API REST deve responder consultas de métricas em menos de 300ms	Desempenho
RNF03	O agente deve ser compatível com Windows 10/11 (obrigatório) e Linux Ubuntu 22.04+ (desejável)	Compatibilidade
RNF04	O sistema deve suportar no mínimo 20 endpoints ativos simultaneamente sem degradação perceptível	Escalabilidade
RNF05	A comunicação entre agente e servidor deve ser autenticada para evitar envio não autorizado de dados	Segurança
RNF06	O dashboard deve ser responsivo e funcional em navegadores modernos (Chrome, Firefox, Edge)	Usabilidade
RNF07	O sistema deve permitir configuração do intervalo de coleta (candidato: 60 segundos)	Configurabilidade
RNF08	Os dados históricos devem ser retidos por no mínimo 30 dias	Armazenamento
RNF09	O servidor central deve ser executável em hardware modesto, sem requisitos de infraestrutura cloud	Portabilidade

2.4 Casos de Uso Principais

ID	Nome	Ator	Descrição
UC01	Registrar endpoint	Agente	O agente realiza o primeiro envio de dados ao servidor, que cadastra automaticamente o endpoint com um identificador único (hostname + IP).
UC02	Enviar métricas	Agente	Em intervalos regulares, o agente coleta métricas do sistema e as envia ao servidor central via HTTP POST. O servidor valida e persiste os dados.
UC03	Enviar evento de arquivo	Agente	Ao detectar criação, modificação ou exclusão de arquivo em diretório monitorado, o agente envia o evento ao servidor com timestamp e metadados.
UC04	Visualizar painel geral	Administrador	Administrador acessa o dashboard e visualiza a lista de todos os endpoints com status (online/offline/alerta), última coleta e métricas resumidas.
UC05	Detalhar endpoint	Administrador	Administrador seleciona um endpoint específico e visualiza métricas detalhadas em tempo quase-real e gráficos históricos de série temporal.
UC06	Visualizar eventos de arquivo	Administrador	Administrador consulta o registro de eventos detectados nos diretórios monitorados de um endpoint, com filtros por data.
UC07	Receber alerta visual	Administrador	Ao acessar o dashboard, o administrador visualiza alertas visuais nos endpoints que ultrapassaram thresholds configurados (ex.: CPU > 90%).



2.5 Regras de Negócio

ID	Regra
RN01	O endpoint é identificado de forma única pela combinação de hostname e endereço IP. Dois agentes com o mesmo hostname mas IPs diferentes são tratados como endpoints distintos.
RN02	Um endpoint é considerado inativo (offline) quando não envia dados ao servidor por um período superior a 3 ciclos consecutivos do intervalo de coleta configurado.
RN03	Um endpoint entra em estado de alerta quando ao menos uma métrica monitorada ultrapassa o threshold definido. O estado de alerta persiste até que a métrica retorne ao nível normal em pelo menos 2 coletas consecutivas.
RN04	O agente só pode enviar dados ao servidor se estiver autenticado com um token ou chave previamente configurada. Requisições sem autenticação válida são rejeitadas.
RN05	O intervalo de coleta é configurado globalmente no servidor. Todos os agentes devem respeitar esse intervalo, consultando a configuração ao inicializar.
RN06	Os dados históricos são retidos por no mínimo 30 dias. Registros mais antigos podem ser removidos automaticamente por uma rotina de limpeza configurável.
RN07	Os diretórios monitorados por cada agente são configurados no arquivo de configuração local do agente, não podendo ser alterados remotamente pelo dashboard nesta versão.

2.6 Fora do Escopo

O sistema não contempla, nesta versão, as seguintes funcionalidades:

- Envio de notificações externas ao dashboard (e-mail, SMS, webhook) — alertas são exibidos apenas na interface web.
- Controle de acesso por usuário — o dashboard não possui sistema de login ou perfis de acesso na versão inicial.
- Monitoramento de processos individuais em execução nos endpoints.
- Gerenciamento remoto dos endpoints a partir do dashboard (reinicialização, execução de comandos).
- Suporte a macOS como sistema operacional do agente.
- Integração com provedores de nuvem ou sistemas SNMP existentes.

3. Fluxos e Comportamento do Sistema

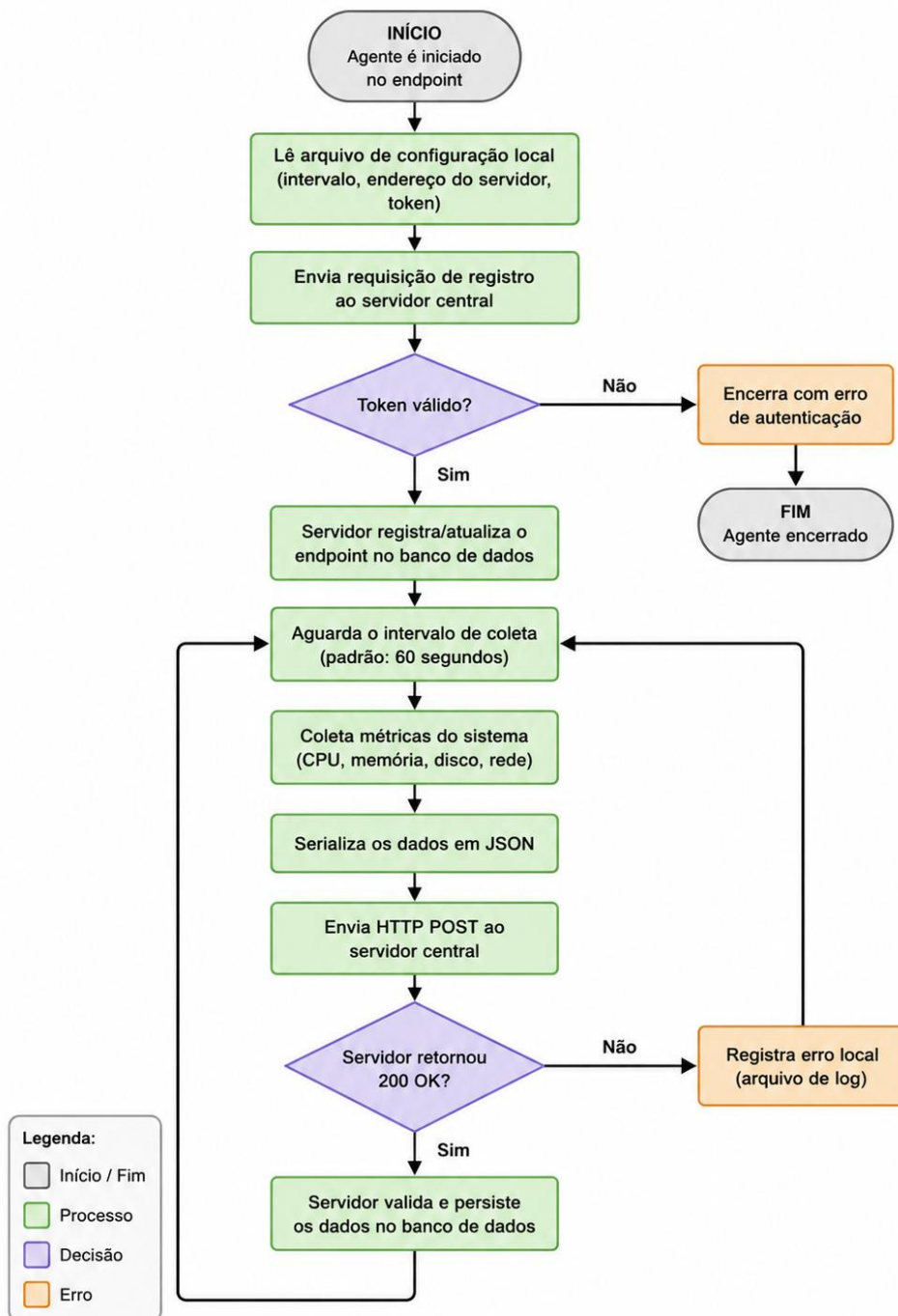
3.1 Fluxo de Coleta e Envio de Métricas

Descrição

O agente é inicializado no endpoint, lê o arquivo de configuração local (intervalo de coleta, endereço do servidor, token de autenticação e diretórios monitorados), registra o endpoint no servidor e inicia o ciclo contínuo de coleta e envio de dados.

Passo a passo

- Agente é iniciado no endpoint (manualmente ou como serviço do sistema operacional)
- Agente lê o arquivo de configuração local (config.json / config.ini)
- Agente envia requisição de registro ao servidor central com hostname, IP e metadados do sistema
- Servidor valida o token de autenticação e registra ou atualiza o endpoint no banco de dados
- Agente inicia o loop de coleta: aguarda o intervalo configurado (padrão: 60 segundos)
- Agente coleta métricas: percentual de CPU, memória usada/total, espaço em disco usado/total por partição, bytes enviados/recebidos por interface de rede
- Agente serializa as métricas em JSON e envia ao servidor via HTTP POST
- Servidor recebe, valida e persiste os dados. Retorna 200 OK ou código de erro
- Agente registra o resultado e aguarda o próximo ciclo



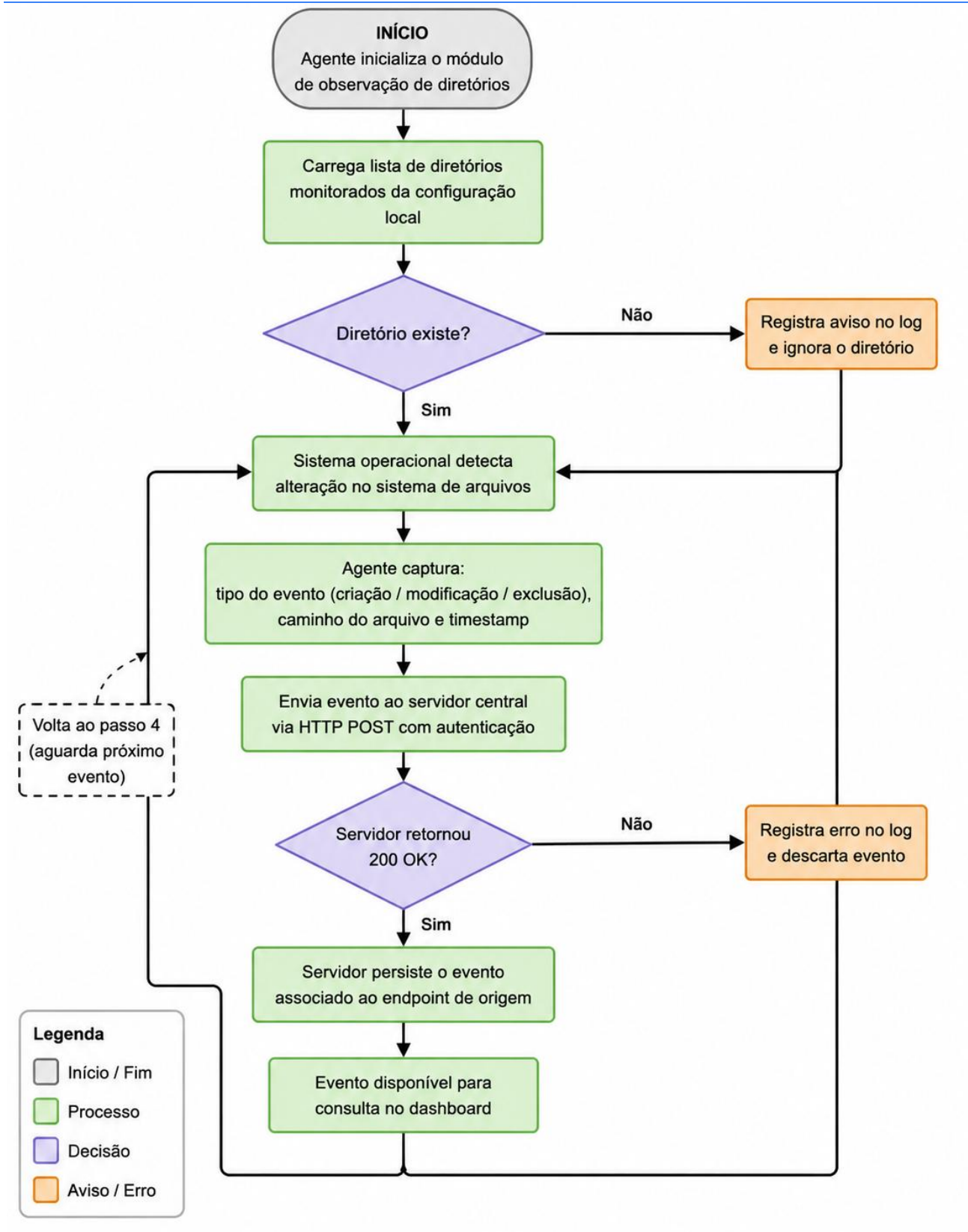
3.2 Fluxo de Detecção de Eventos em Sistema de Arquivos

Descrição

Em paralelo ao ciclo de coleta de métricas, o agente mantém um processo de observação contínua dos diretórios configurados. Ao detectar qualquer alteração (criação, modificação ou exclusão de arquivo), o agente imediatamente envia o evento ao servidor, independente do ciclo de coleta.

Passo a passo

- Agente inicializa o módulo de observação de diretórios com a lista de caminhos configurados
- Sistema operacional notifica o agente sobre alteração no sistema de arquivos (via watchdog / inotify / ReadDirectoryChangesW)
- Agente captura o tipo de evento (criação / modificação / exclusão), caminho do arquivo e timestamp
- Agente envia o evento ao servidor central via HTTP POST com autenticação
- Servidor valida e persiste o evento, associando-o ao endpoint de origem
- Evento fica disponível para consulta no dashboard via API REST



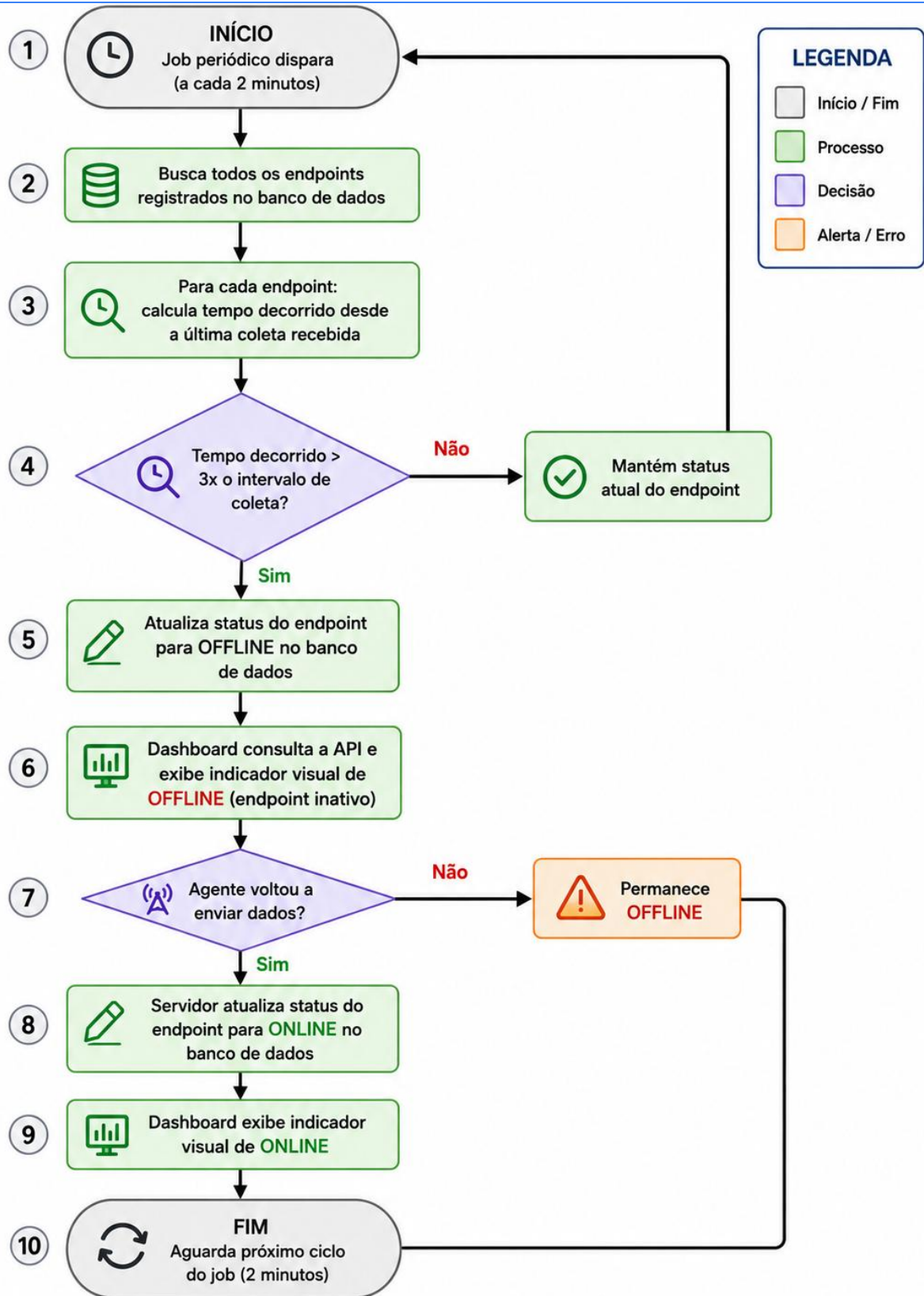
3.3 Fluxo de Detecção de Endpoint Inativo

Descrição

O servidor central mantém o registro do timestamp da última coleta recebida de cada endpoint. Um job periódico verifica se algum endpoint não enviou dados dentro do tempo esperado, alterando seu status para offline no banco de dados.

Passo a passo

- Job periódico no servidor executa a cada intervalo de verificação (ex.: a cada 2 minutos)
- Para cada endpoint registrado, verifica o timestamp da última coleta recebida
- Se o tempo decorrido desde a última coleta for superior a 3x o intervalo de coleta configurado, o endpoint é marcado como offline
- Dashboard consulta o status atualizado via API e exibe o indicador visual correspondente (offline / alerta)
- Quando o agente voltar a enviar dados, o servidor atualiza automaticamente o status para online



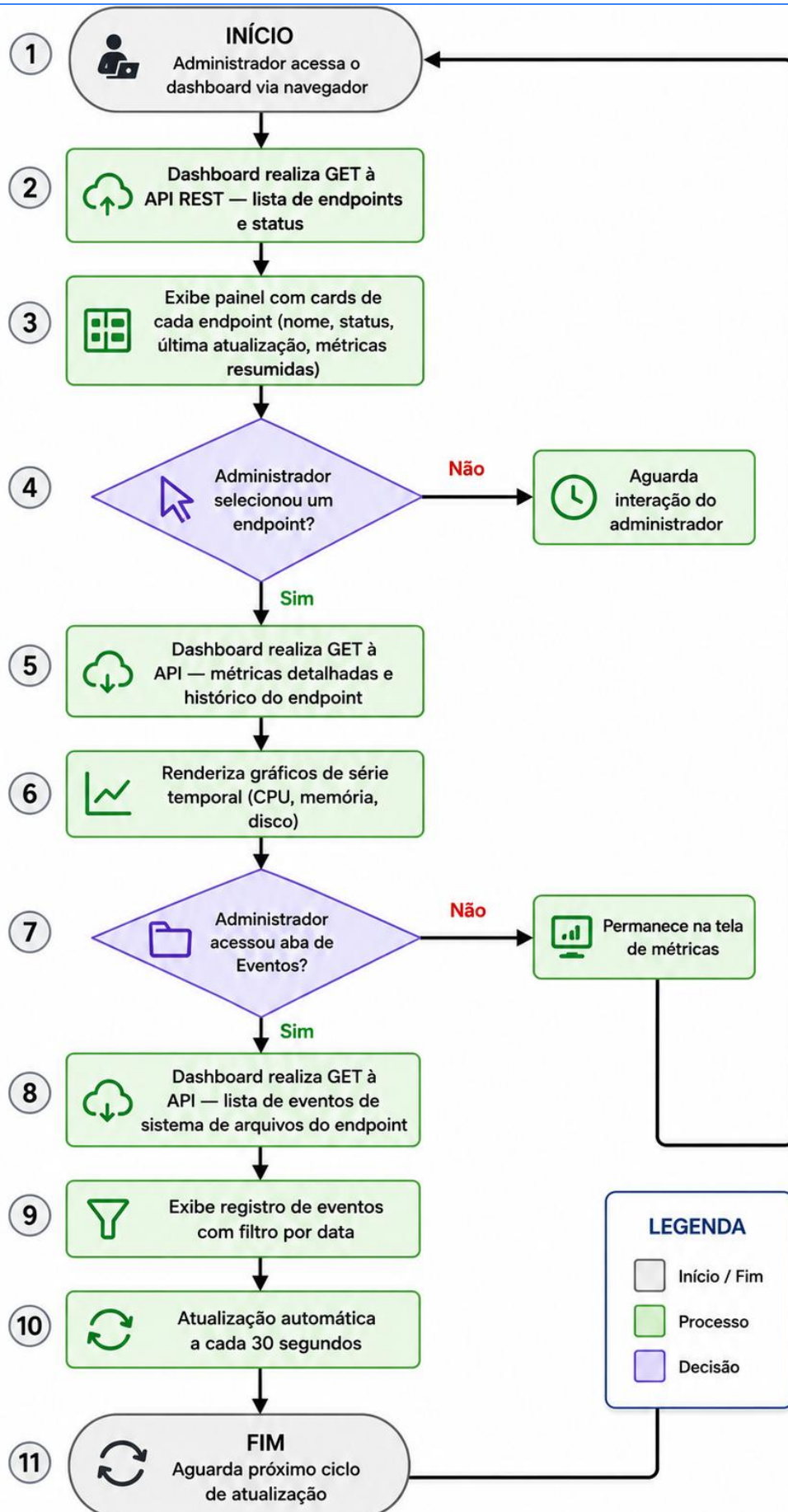
3.4 Fluxo de Visualização no Dashboard

Descrição

O administrador acessa o dashboard web pelo navegador. A interface consulta periodicamente a API REST do servidor central e atualiza a exibição com os dados mais recentes dos endpoints.

Passo a passo

- Administrador acessa o dashboard via navegador (ex.: `http://servidor:3000`)
- Dashboard realiza requisição GET à API REST para obter a lista de endpoints e seus status
- Administrador visualiza o painel com cards de cada endpoint indicando: nome, status, última atualização e métricas resumidas
- Ao selecionar um endpoint, o dashboard realiza nova requisição para obter as métricas detalhadas e o histórico
- Dashboard renderiza gráficos de série temporal (CPU, memória, disco) com os dados históricos retornados
- Administrador pode navegar para a aba de Eventos para visualizar os registros de sistema de arquivos do endpoint selecionado
- Dashboard atualiza automaticamente os dados a cada 30 segundos sem necessidade de recarregar a página



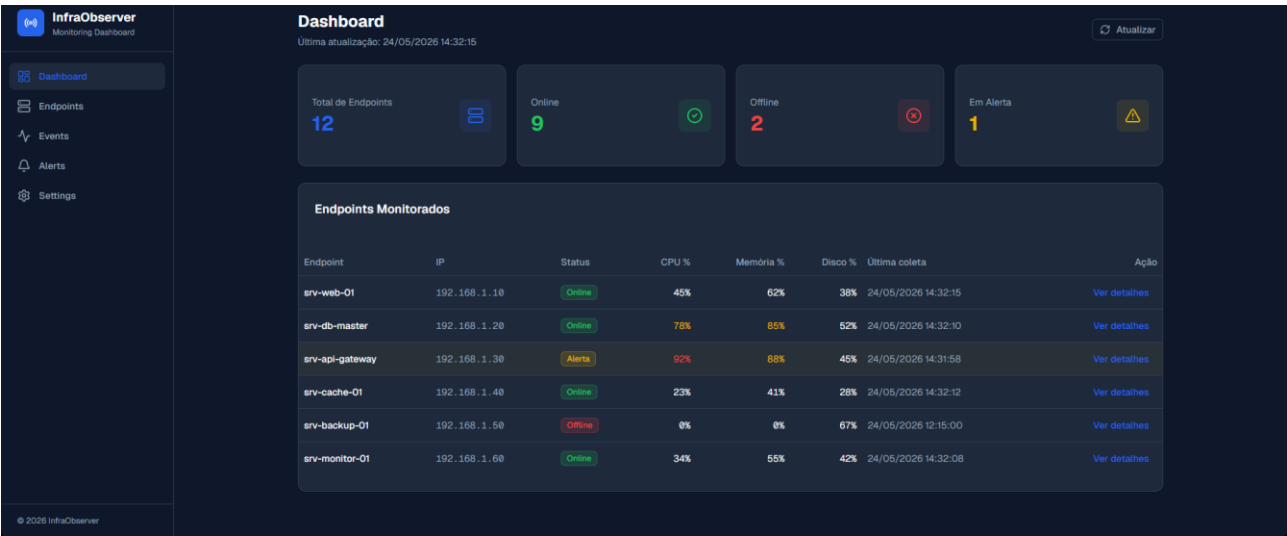
4.1 Fluxo de Navegação

O fluxo principal de navegação do administrador no dashboard segue a estrutura:

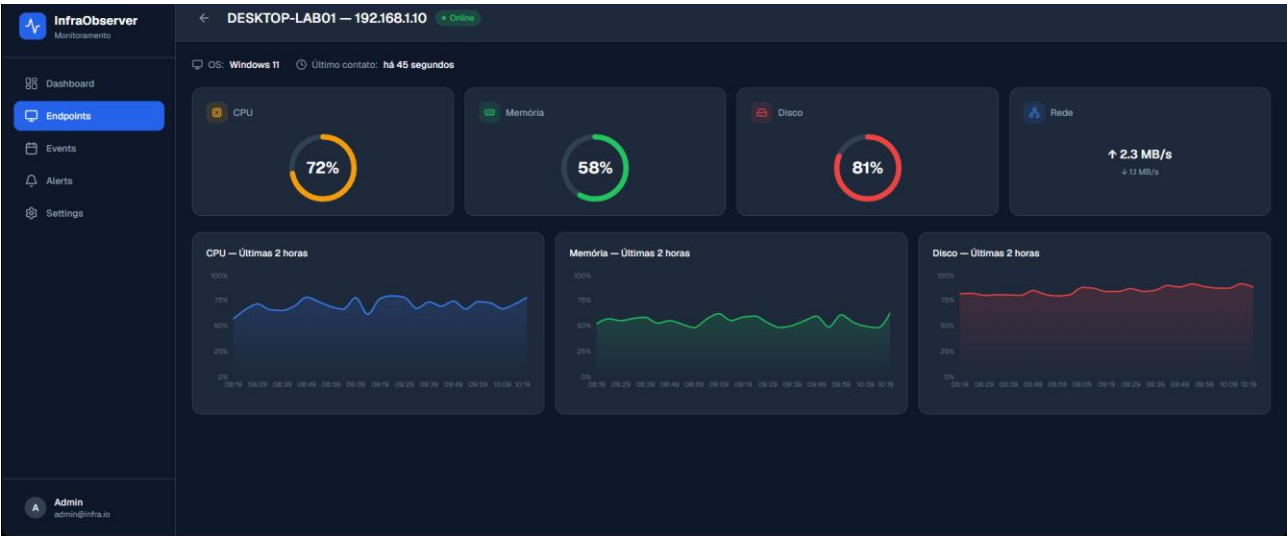


4.2 Wireframes e Mockups das Telas

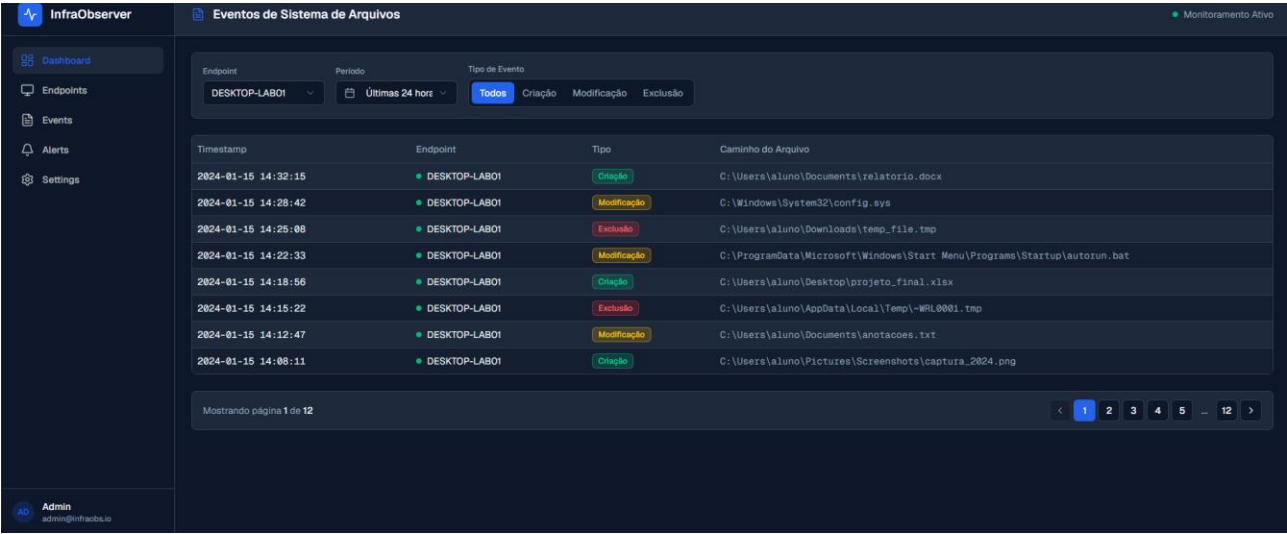
Tela 1 — Dashboard Geral (Lista de Endpoints)



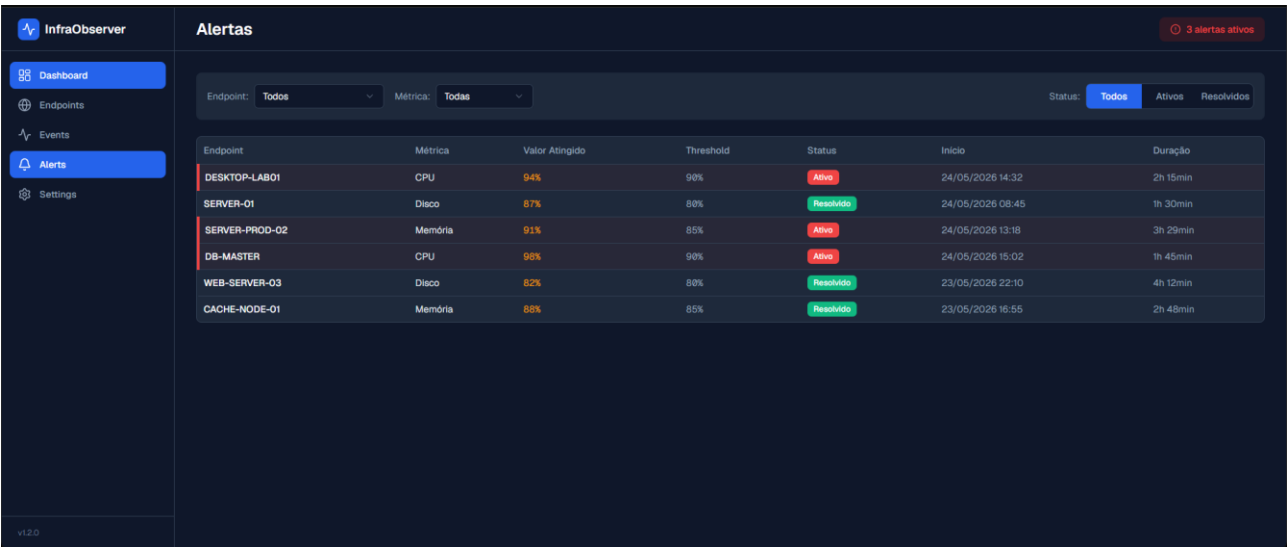
Tela 2 — Detalhe do Endpoint (Métricas e Gráficos)



Tela 3 — Registro de Eventos de Sistema de Arquivos



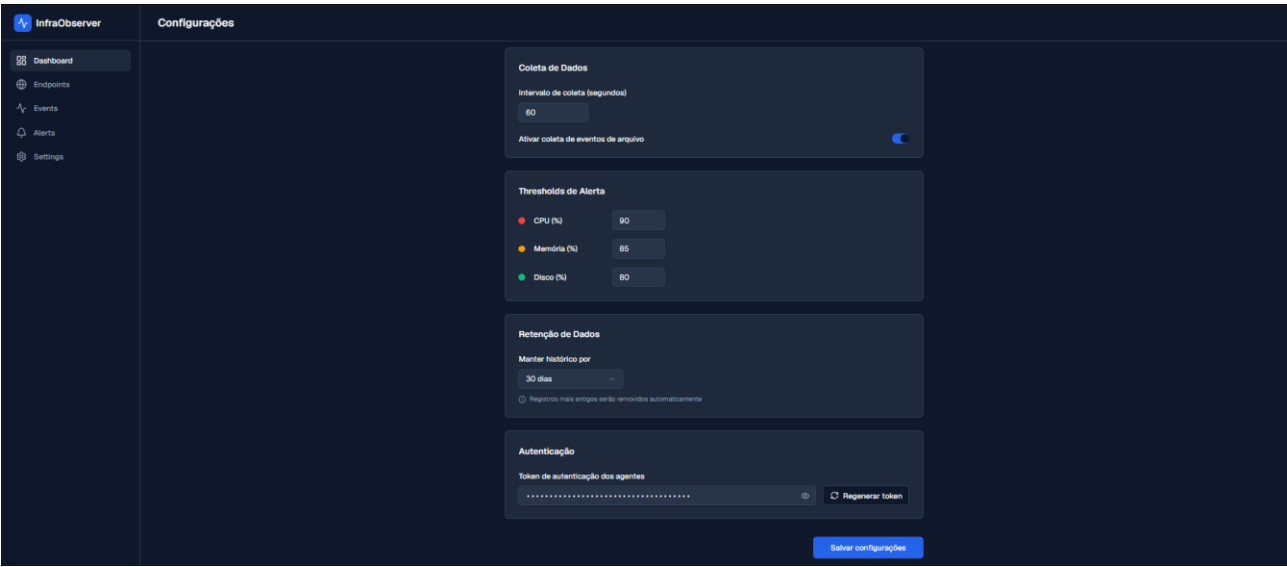
Tela 4 — Página de Alertas



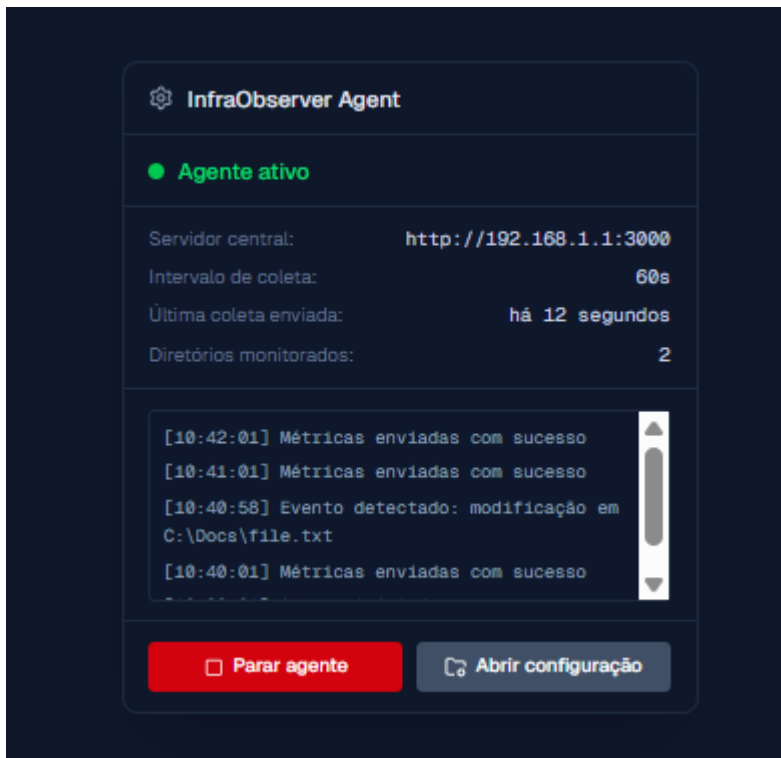
Tela 5 — Lista de Endpoints (Gerenciamento)



Tela 6 — Configurações do Sistema



Tela 7 — Agente: Tela de Status (instalado no endpoint)

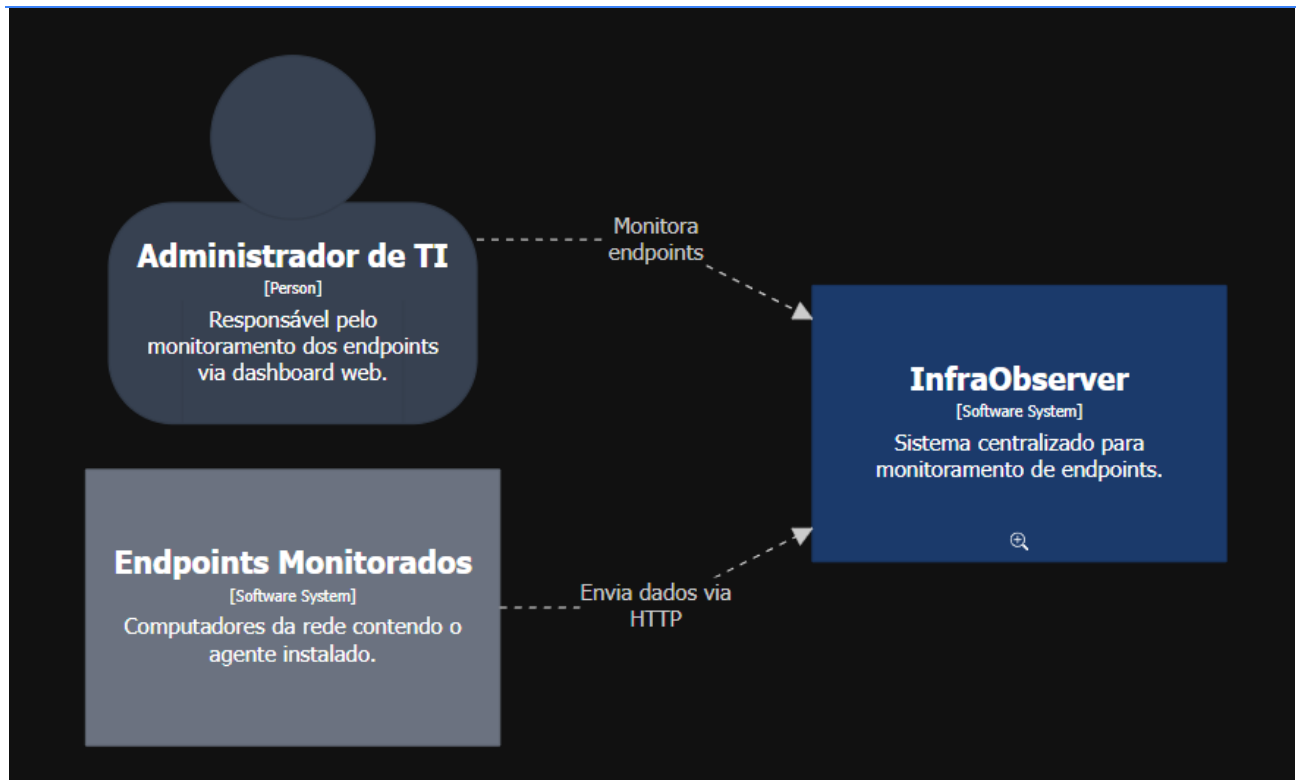


5. Arquitetura do Sistema

5.1 Diagrama C4

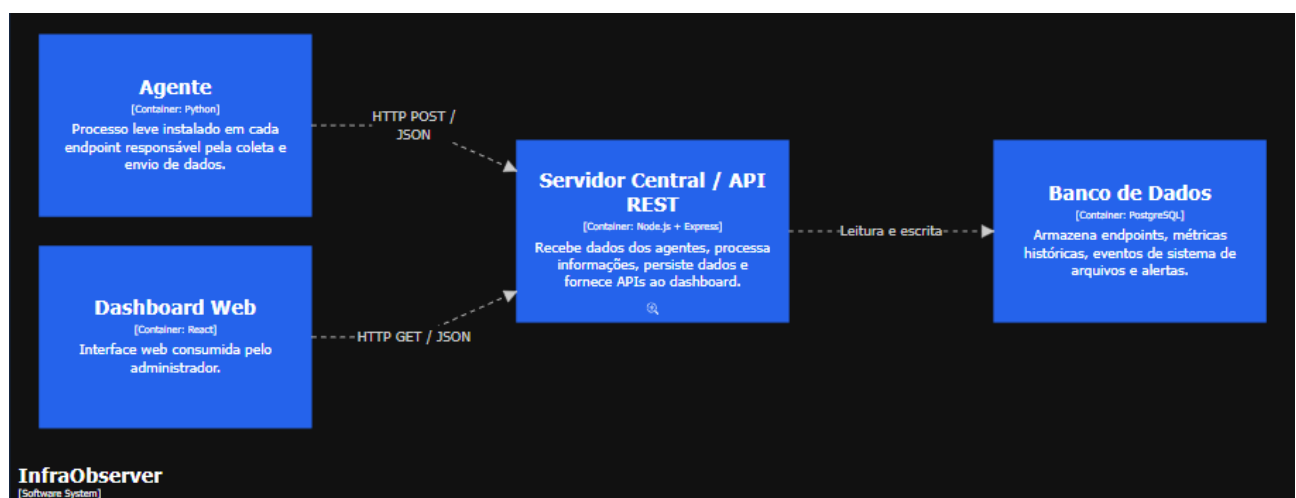
Nível 1 — Diagrama de Contexto

O InfraObserver é composto por três elementos principais no contexto do sistema: os endpoints monitorados (com o agente instalado), o servidor central (backend da aplicação) e o administrador que acessa o dashboard. Os endpoints se comunicam com o servidor via HTTP. O administrador acessa o dashboard via navegador web.



Nível 2 — Diagrama de Containers

O sistema é dividido em três containers principais: o Agente (processo Python instalado nos endpoints), o Servidor Central (API REST + banco de dados) e o Dashboard Web (interface frontend). A comunicação entre Agente e Servidor é feita via HTTP/JSON. O Dashboard consome a API REST do Servidor.



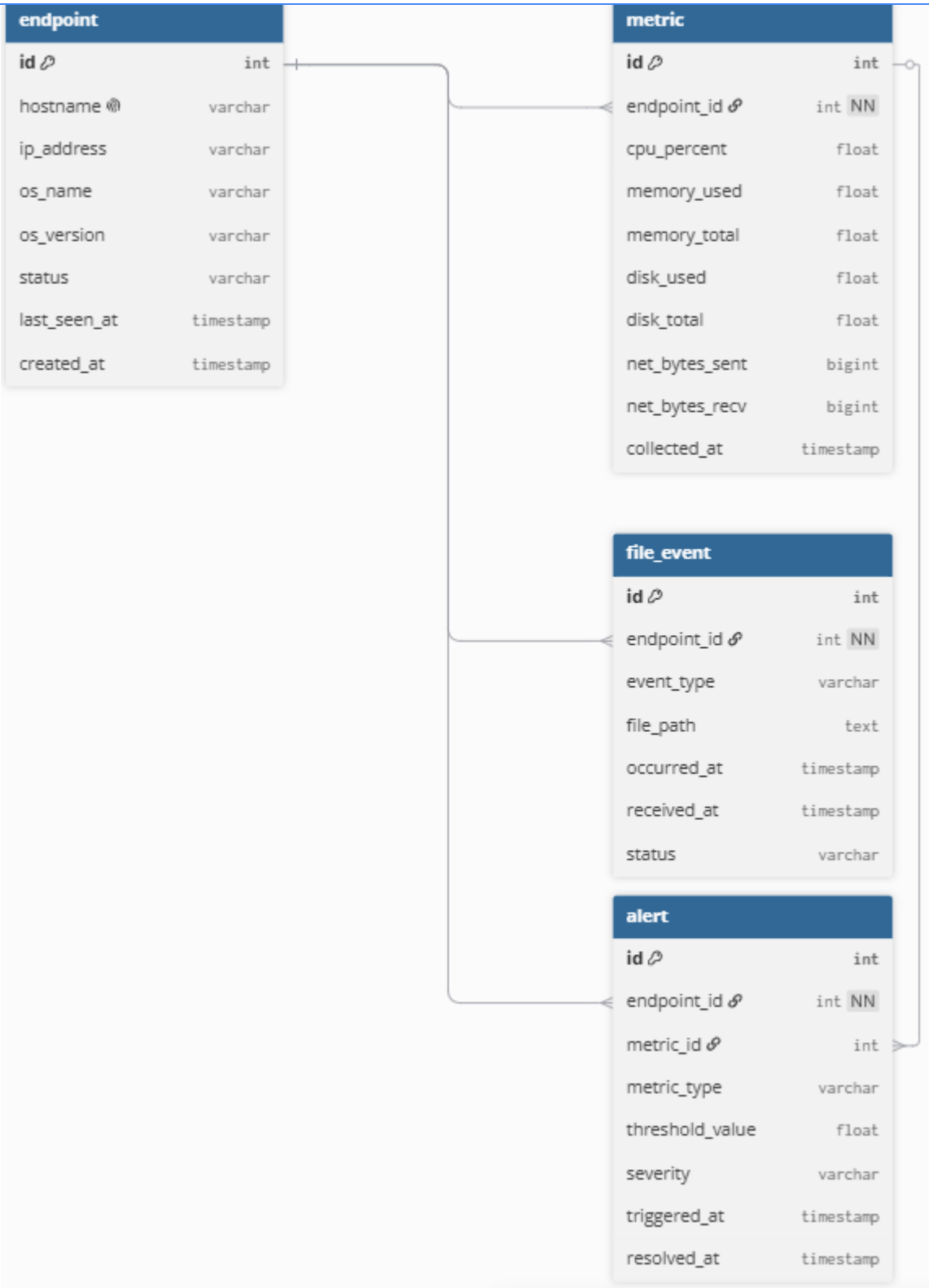
Nível 3 — Diagrama de Componentes (Servidor Central)

O Servidor Central é organizado em camadas, seguindo a separação de responsabilidades. As requisições dos agentes e do dashboard passam pelo Router, são processadas pelos Controllers, que delegam à camada de Services para a lógica de negócio, e à camada de Repositories para persistência.



5.2 Modelo de Dados

O banco de dados é composto por quatro entidades principais. O diagrama Entidade-Relacionamento conceitual é apresentado abaixo:



5.3 Principais Componentes

Componente	Tecnologia	Responsabilidade
Agente (Agent)	Python	Processo leve instalado em cada endpoint. Responsável por coletar métricas via psutil, detectar eventos de arquivo via watchdog e enviar os dados ao servidor via HTTP/JSON. Configurável por arquivo local.
API REST (Backend)	Node.js + Express	Servidor central que expõe os endpoints da API. Responsável por autenticar requisições dos agentes e do dashboard, validar os dados recebidos e delegar ao Service Layer.
Service Layer	Node.js	Camada de lógica de negócio. Processa as métricas recebidas, executa o job de verificação de inatividade dos endpoints, gera alertas e agrega os dados para o dashboard.
Repository Layer	Node.js + ORM	Camada de acesso a dados. Abstrai as operações de leitura e escrita no banco de dados, isolando a lógica de persistência dos demais componentes.
Banco de Dados	SQLite / PostgreSQL	Armazena endpoints registrados, histórico de métricas, eventos de arquivo e alertas gerados. SQLite para desenvolvimento e ambientes simples; PostgreSQL para produção.
Dashboard Web	React / Vue.js	Interface web para o administrador. Consome a API REST via polling periódico. Exibe cards de status dos endpoints, gráficos de série temporal e listagem de eventos de arquivo.

5.4 Stack Tecnológica

Tecnologia	Camada	Justificativa
Python 3.11+	Agente	Linguagem amplamente suportada em Windows e Linux. Biblioteca psutil oferece API unificada para coleta de métricas do sistema operacional. watchdog para monitoramento de sistema de arquivos.
psutil	Agente	Biblioteca Python multiplataforma para coleta de métricas de sistema (CPU, memória, disco, rede) com API simples e sem necessidade de permissões elevadas para uso básico.
watchdog	Agente	Biblioteca Python para monitoramento de eventos em sistema de arquivos. Suporte nativo a inotify (Linux) e ReadDirectoryChangesW (Windows).

Tecnologia	Camada	Justificativa
Node.js + Express	Servidor Central	Runtime JavaScript de alta performance para I/O assíncrono. Express é um framework minimalista que facilita a construção de APIs REST sem overhead desnecessário.
SQLite / PostgreSQL	Servidor Central	SQLite para ambientes de desenvolvimento e laboratório (zero configuração). PostgreSQL para cenários com maior volume de dados ou múltiplos acessos simultâneos.
React ou Vue.js	Dashboard Web	Frameworks JavaScript modernos para construção de interfaces reativas. A escolha final será definida durante o desenvolvimento com base na familiaridade do autor.
Recharts / Chart.js	Dashboard Web	Bibliotecas de gráficos para React/Vue. Utilizadas para renderizar os gráficos de série temporal das métricas históricas.